Experiment -1

AIM : Write a program in MATLAB to solve homogeneous and non-homogeneous linear system of differential equations using matrix method.

THEORY :

A **linear system of differential equations** can be written as:

✅ Homogeneous System:

$$\frac{d\mathbf{x}}{dt} = A\mathbf{x}$$

Solution:

$$\mathbf{x}(t) = e^{At}\mathbf{x}_0$$

✅ Non-Homogeneous System:

$$\frac{d\mathbf{x}}{dt} = A\mathbf{x} + \mathbf{f}(t)$$

General Solution:

$$\mathbf{x}(t) = e^{At}\mathbf{x}_0 + \int_0^t e^{A(t-\tau)}\mathbf{f}(\tau)d\tau$$

CODE - 1 :

For Homogenous System –

```
function [sols] = linear_system_DE_(A)
   syms t lambda
   A = [ 6  5; 1 2];
   n = length(A);
   [V, D] = eig(A);
   eigenvalues = diag(D);
```

```
  const = reshape(sym('c%d', [1 n]), n, 1);
  unique_eigenvalues = unique(eigenvalues);
  mults = histcounts(eigenvalues, unique_eigenvalues);
  sols = sym('x%d', [1 n]);
    i = 1;
  if length(unique_eigenvalues) ~= length(eigenvalues)
    ch_mat = A - lambda * eye(n);
    V = vpa(V);
    while i <= n
      [pos] = find(unique_eigenvalues == eigenvalues(i));
      if mults(pos) > i
        e_vector = V(:, i);
        a_mat = subs(ch_mat, eigenvalues(i));
                for j = 1:mults(pos)
          V(:, i) = V(:, i) .* (t^(j - 1));
          P = (a_mat^(j - 1)) * e_vector;
          V(:, i) = V(:, i) + P;
          i = i + 1;
        end
      else
        i = i + 1;
      end
    end
  end
  for i = 1:n
    sols(i) = (V(i, :) * exp(eigenvalues .* t)) * const(i);
  end
end
```

OUTPUT :

```
A = 2×2
     6     5
     1     2
```

```
sols =
```

$$\left(-c_1\left(\frac{\sqrt{2}\,e^{t}}{2}-\frac{5\sqrt{26}\,e^{7t}}{26}\right)\quad x_2\right)$$

```
sols =
```

$$\left(-c_1\left(\frac{\sqrt{2}\,e^{t}}{2}-\frac{5\sqrt{26}\,e^{7t}}{26}\right)\quad c_2\left(\frac{\sqrt{2}\,e^{t}}{2}+\frac{\sqrt{26}\,e^{7t}}{26}\right)\right)$$

CODE - 2 :

For Non Homogenous System –

```
function [sols] = linear_DE_System(~)
   syms t lambda
   % Input the system matrix and the non-homogeneous term
   A = input('Enter the coefficient matrix A: ');
   F = input('Enter the non-homogeneous term (vector or matrix) F(t): ');
   n = length(A);
% Solve for eigenvalues and eigenvectors
   [V, D] = eig(A);
   eigenvalues = diag(D);
   const = reshape(sym('c%d', [1 n]), n, 1);
   unique_eigenvalues = unique(eigenvalues);
```

```
  mults = histcounts(eigenvalues, unique_eigenvalues); % Count multiplicities
  sols = sym('x%d', [1 n]);
  generalized_V = sym(zeros(n, n)); % Store all eigenvectors and generalized
eigenvectors
  i = 1;
% Handle repeated eigenvalues
  if length(unique_eigenvalues) ~= length(eigenvalues)
    for idx = 1:length(unique_eigenvalues)
    lambda_i = unique_eigenvalues(idx);
    mult = mults(idx);
    ch_mat = A - lambda_i * eye(n);
% Compute eigenvectors and generalized eigenvectors
      for j = 1:mult
        if j == 1
% Basic eigenvector
          e_vector = V(:, i);
        else

          e_vector = (ch_mat^(j-1))
          generalized_V(:, i + j - 2);
        end
% Scale by t^(j-1)
        generalized_V(:, i + j - 1) = e_vector * t^(j-1);
      end
      i = i + mult;
    end
  else
    generalized_V = V;
  end
% Homogeneous solution
```

```
generalized_V
hom_sols = sym(zeros(n, 1));
for k = 1:n
   hom_sols = hom_sols + exp(eigenvalues(k) * t) * (generalized_V(:, k).' * const);
end
hom_sols
W = generalized_V * exp(D * t) % Fundamental matrix
inv_W = simplify(inv(W)) % Inverse of fundamental matrix
int_term = int(inv_W * F, t) % Integral of (W^-1 * F)
part_sols = simplify(W * int_term) % Particular solution

% Combine homogeneous and particular solutions
sols = simplify(hom_sols + part_sols);
```

OUTPUT :

A = 2×2

	1	2
1	5	2
2	-7	-4

F =

$$\begin{pmatrix} 2\,e^{t} \\ 4\,e^{t} \end{pmatrix}$$

V = 2×2

```
    0.7071   -0.2747
   -0.7071    0.9615
```

D = 2×2

```
     3     0
     0    -2
```

```
generalized_V = 2×2
      0.7071   -0.2747
     -0.7071    0.9615
```

hom_sols =

$$\begin{pmatrix}\sigma_1\\ \sigma_1\end{pmatrix}$$

where

$$\sigma_1 = e^{3t}\left(\frac{\sqrt{2}\,c_1}{2} - \frac{\sqrt{2}\,c_2}{2}\right) - e^{-2t}\left(\frac{2\sqrt{53}\,c_1}{53} - \frac{7\sqrt{53}\,c_2}{53}\right)$$

W =

$$\begin{pmatrix}\frac{\sqrt{2}\,e^{3t}}{2} - \frac{2\sqrt{53}}{53} & \frac{\sqrt{2}}{2} - \frac{2\sqrt{53}\,e^{-2t}}{53}\\ \frac{7\sqrt{53}}{53} - \frac{\sqrt{2}\,e^{3t}}{2} & \frac{7\sqrt{53}\,e^{-2t}}{53} - \frac{\sqrt{2}}{2}\end{pmatrix}$$

sols =

$$\begin{pmatrix}\sigma_2 - \sigma_1 + \left(\frac{\sqrt{2}}{2} - \frac{2\sqrt{53}\,e^{-2t}}{53}\right)\sigma_4 - \sigma_5\left(\frac{2\sqrt{53}}{53} - \sigma_3\right)\\ \sigma_2 - \sigma_1 - \left(\frac{\sqrt{2}}{2} - \frac{7\sqrt{53}\,e^{-2t}}{53}\right)\sigma_4 + \sigma_5\left(\frac{7\sqrt{53}}{53} - \sigma_3\right)\end{pmatrix}$$

where

$$\sigma_1 = e^{-2t}\left(\frac{2\sqrt{53}\,c_1}{53} - \frac{7\sqrt{53}\,c_2}{53}\right)$$

$$\sigma_2 = e^{3t}\left(\frac{\sqrt{2}\,c_1}{2} - \frac{\sqrt{2}\,c_2}{2}\right)$$

$$\sigma_3 = \frac{\sqrt{2}\,e^{3t}}{2}$$

$$\sigma_4 = \frac{6\sqrt{53}\,e^{t}}{5} - \sigma_6 + \frac{3\sqrt{53}\,e^{2t}}{5} + \frac{2\sqrt{53}\,e^{3t}}{5}$$

$$\sigma_5 = \frac{e^{-t}\left(1166\sqrt{2} - 1166\sqrt{2}\,t\,e^{t}\right)}{265} + \sigma_6$$

$$\sigma_6 = \log(e^{t} - 1)\left(\frac{22\sqrt{2}}{5} - \frac{6\sqrt{53}}{5}\right)$$

Experiment -2

AIM : Write a program in MATLAB to solve Initial Value Problems of homogeneous and non-homogeneous linear system of differential equations using matrix method.

CODE :

```
function [sols] = linear_DE_System_HomN_IVP(~)
  syms t lambda
  % Input the system matrix, non-homogeneous term, and initial conditions
  A = input('Enter the coefficient matrix A: ');
  F = input('Enter the non-homogeneous term (vector or matrix) F(t): ');
  C0 = input('Enter the initial condition vector (e.g., [1; 0; 1]): ');
  n = length(A);
  % Solve for eigenvalues and eigenvectors
  [V, D] = eig(A);
  eigenvalues = diag(D);
  const = sym('c', [n, 1]); % Constants for homogeneous solution
  unique_eigenvalues = unique(eigenvalues);
  mults = histcounts(eigenvalues, unique_eigenvalues); % Count multiplicities
  generalized_V = sym(zeros(n, n)); % Store all eigenvectors and generalized
eigenvectors
  i = 1;
  % Handle repeated eigenvalues
  if length(unique_eigenvalues) ~= length(eigenvalues)
    for idx = 1:length(unique_eigenvalues)
      lambda_i = unique_eigenvalues(idx);
      mult = mults(idx);
      ch_mat = A - lambda_i * eye(n);
      % Compute eigenvectors and generalized eigenvectors
      for j = 1:mult
```

```
        if j == 1
            % Basic eigenvector
            e_vector = V(:, i);
        else
            % Generalized eigenvector
            e_vector = (ch_mat^(j-1)) \ generalized_V(:, i + j - 2);
        end
        % Scale by t^(j-1)
        generalized_V(:, i + j - 1) = e_vector * t^(j-1);
    end
    i = i + mult;
  end
else
  generalized_V = V; % No repeated eigenvalues
end
% Compute the homogeneous solution
hom_sols = sym(zeros(n, 1));
for k = 1:n
  hom_sols = hom_sols + exp(eigenvalues(k) * t) * generalized_V(:, k) * const(k);
end
% Compute the particular solution using variation of parameters
W = generalized_V * diag(exp(eigenvalues * t)); % Fundamental matrix
inv_W = simplify(inv(W)); % Inverse of fundamental matrix
int_term = int(inv_W * F, t); % Integral of (W^(-1) * F)
part_sols = simplify(W * int_term); % Particular solution
% Combine homogeneous and particular solutions
sols = simplify(hom_sols + part_sols);
% Apply initial conditions
sols_at_t0 = subs(sols, t, 0); % Solution at t = 0
```

```
    eqns = sols_at_t0 == C0; % Equations for constants
    vals = solve(eqns, const); % Solve for constants

    % Substitute constants into the solution
    for k = 1:n
        sols = subs(sols, const(k), vals.(sprintf('c%d', k)));
    end
    % Simplify the final solution
    sols = simplify(sols);
end
```

OUTPUT :

```
A = 2×2
     1     6
     1     2
```

```
F =
```

$$\begin{pmatrix} 2t \\ t \end{pmatrix}$$

```
initial_value = 2×1
     1
     0
```

```
vals = struct with fields:
    c1: -10^(1/2)/5
    c2: -(21*5^(1/2))/80
```

```
sols =
```

$$\begin{pmatrix} \frac{3\,e^{-t}}{5} - \frac{t}{2} + \frac{21\,e^{4t}}{40} - \frac{1}{8} \\ \frac{21\,e^{4t}}{80} - \frac{e^{-t}}{5} - \frac{t}{4} - \frac{1}{16} \end{pmatrix}$$

Experiment -3

AIM : Write a program in MATLAB to solve Sturm Liouville boundary value problem to obtain characteristic values and characteristic functions.

THEORY :

A **Sturm–Liouville problem** is a second-order linear differential equation of the form:

$$\frac{d}{dx}\left[p(x)\frac{dy}{dx}\right] + [\lambda w(x) - q(x)]\,y = 0$$

on an interval $[a, b]$, with **boundary conditions**:

$$\alpha_1 y(a) + \alpha_2 y'(a) = 0, \quad \beta_1 y(b) + \beta_2 y'(b) = 0$$

Where:

- λ is the **eigenvalue**,
- $y(x)$ is the **eigenfunction**,
- $p(x), q(x), w(x)$ are given functions with $w(x) > 0$ (weight function),
- α_i, β_i define boundary conditions.

CODE :

```
function sturm_liouville_solver
  a = 0;  % Left boundary
  b = pi/2;  % Right boundary
  N = 100;  % Number of points
  h = (b - a) / (N + 1);  % Step size
  x = linspace(a, b, N+2);  % Discretized domain
  % Define functions
  p = @(x) 1;   % p(x)
  q = @(x) 0;   % q(x)
  w = @(x) x;   % Weight function w(x)
  A = zeros(N, N);
  for i = 1:N
```

```
    xi = x(i+1); % Exclude boundaries
    A(i, i) = -2 * p(xi) / h^2 + q(xi);
    if i > 1
        A(i, i-1) = p(xi) / h^2;
    end
    if i < N
        A(i, i+1) = p(xi) / h^2;
    end
end
    % Solve the eigenvalue problem A * Y = λ * W * Y
[V, D] = eig(A)
lambda = diag(D);
[lambda, idx] = sort(lambda);
V = V(:, idx);
    % Display eigenvalues
fprintf('First 5 Characteristic Values (Eigenvalues):\n');
disp(lambda(1:5));
figure;
hold on;
for k = 1:3
    y = [0; V(:, k); 0];  % Include boundary conditions
    plot(x, y, 'LineWidth', 2);
end
hold off;
xlabel('x');
ylabel('Eigenfunction y(x)');
title('First Three Characteristic Functions');
legend('1st Mode', '2nd Mode', '3rd Mode');
grid on;
```

end

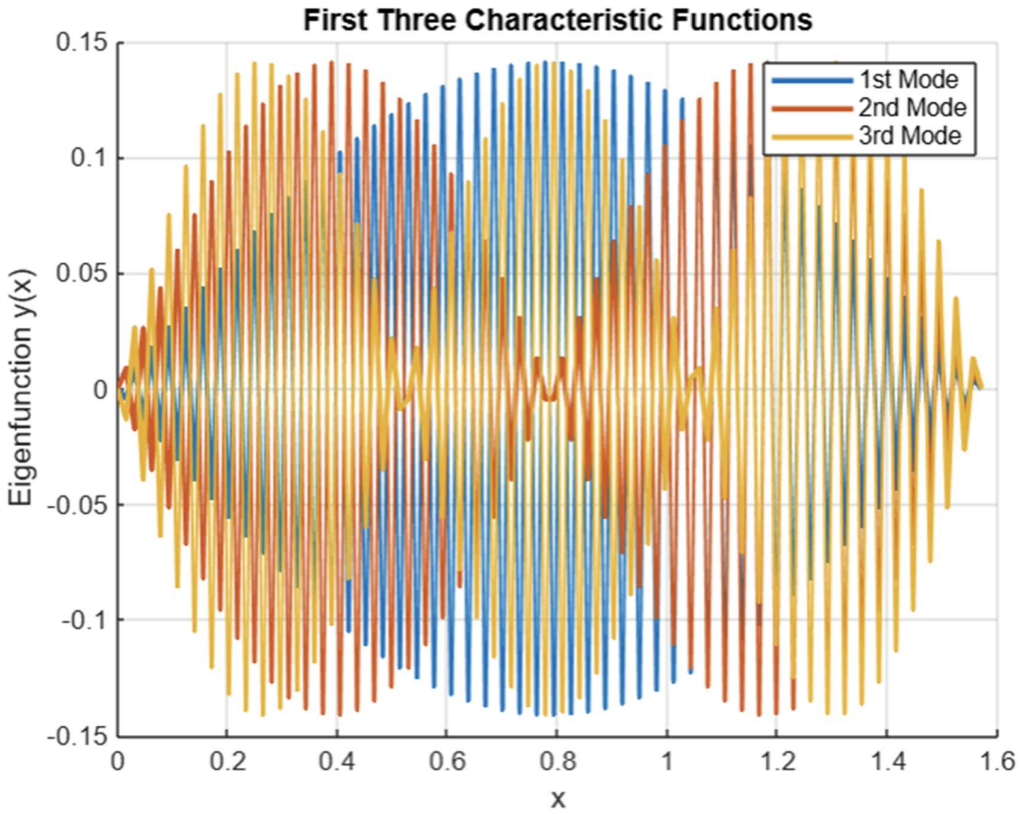
OUTPUT :

```
domain = 1×2
         0    1.5708
```

First 5 Characteristic Values (Eigenvalues):

```
disp(lambda(1:5))
```

```
    1.0e+04 *

   -1.6533
   -1.6521
   -1.6501
   -1.6473
   -1.6437
```



Experiment -4

AIM : Write a program in MATLAB to analyze the stability of linear and non-linear differential equations through phase portrait diagram.

THEORY :

A **phase portrait** is a plot of trajectories of a dynamical system in the phase plane. Each point represents a state (e.g., x and y), and arrows or curves show how the system evolves over time.

- General System:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x})$$

- Linear system: $\mathbf{f}(\mathbf{x}) = A\mathbf{x}$
- Non-linear system: $\mathbf{f}(\mathbf{x})$ has nonlinear terms

---

Stability

- **Stable (sink)**: Trajectories approach equilibrium point
- **Unstable (source)**: Trajectories move away from equilibrium
- **Saddle point**: Some trajectories approach, some diverge
- **Center**: Closed orbits, neutrally stable

CODE :

```
clc; clear; close all;


A = [0 1; -5 0];

f = @(t, x) A * x;
```

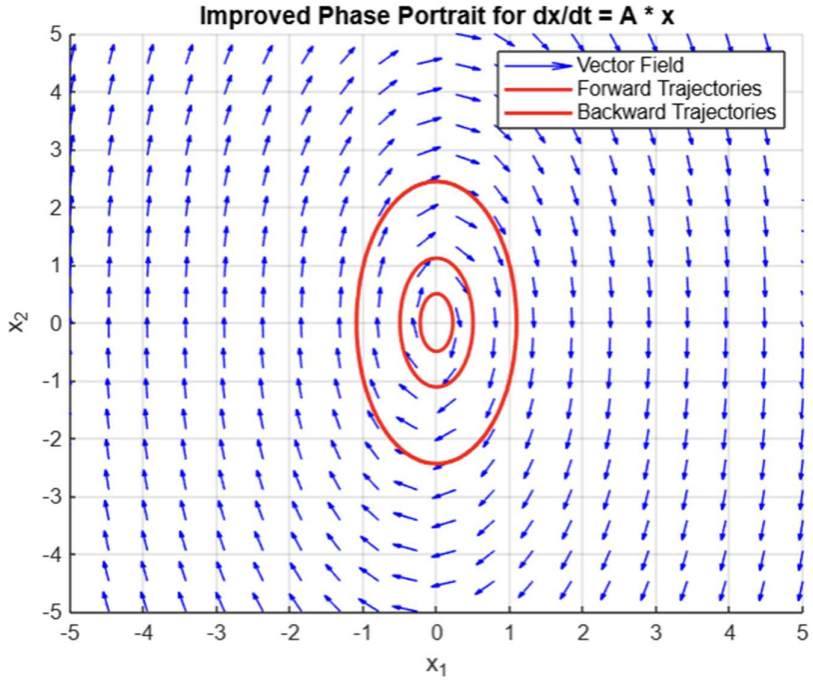
```
[x, y] = meshgrid(linspace(-5, 5, 20), linspace(-5, 5, 20));
u = zeros(size(x));
v = zeros(size(y));

for i = 1:numel(x)
    dx = f(0, [x(i); y(i)]);
    norm_factor = max(norm(dx), 1e-3);
    u(i) = dx(1) / norm_factor;
    v(i) = dx(2) / norm_factor;
end

% Plot vector field
figure; hold on;
quiver(x, y, u, v, 'b', 'AutoScale', 'on', 'AutoScaleFactor', 0.5);

% Detect if the system is expanding
eigenvalues = eig(A)
is_expanding = any(real(eigenvalues) > 0);
if is_expanding
    tspan_fwd = [0, 1];  % Shorter time for unstable cases
    tspan_bwd = [0, -1];
else
    tspan_fwd = [0, 10];
    tspan_bwd = [];
end

% Initial conditions
init_conditions = [-1 -1; 1 1; -1 1; 1 -1; -0.5 0; 0.5 0; 0 0.5; 0 -0.5];
```

```
function [value, isterminal, direction] = stopAtBoundary(~, X)
  value = max(abs(X)) - 5;
  isterminal = 1;
  direction = 0;
end
options = odeset('Events', @stopAtBoundary);

for i = 1:size(init_conditions, 1)
  [T_fwd, X_fwd] = ode45(f, tspan_fwd, init_conditions(i, :)', options);
  plot(X_fwd(:,1), X_fwd(:,2), 'r', 'LineWidth', 1.5);

  if is_expanding
    [T_bwd, X_bwd] = ode45(f, tspan_bwd, init_conditions(i, :)', options);
    plot(X_bwd(:,1), X_bwd(:,2), 'g', 'LineWidth', 1.5);
  end
end

title('Improved Phase Portrait for dx/dt = A * x');
xlabel('x_1'); ylabel('x_2');
grid on; axis([-5 5 -5 5]);
legend('Vector Field', 'Forward Trajectories', 'Backward Trajectories');
eigs = eigenvalues;
if all(real(eigs) < 0)
  disp('System is Stable (Asymptotically Stable)');
elseif any(real(eigs) > 0)
  disp('System is Unstable');
elseif all(real(eigs) == 0)
  disp('System is Marginally Stable (i.e. stable but not asymptotically ) ');
end
```

OUTPUT :: 1::

```
A = 2×2
     0     1
    -5     0
```

```
eigenvalues = 2×1 complex
   0.0000 + 2.2361i
   0.0000 - 2.2361i
```

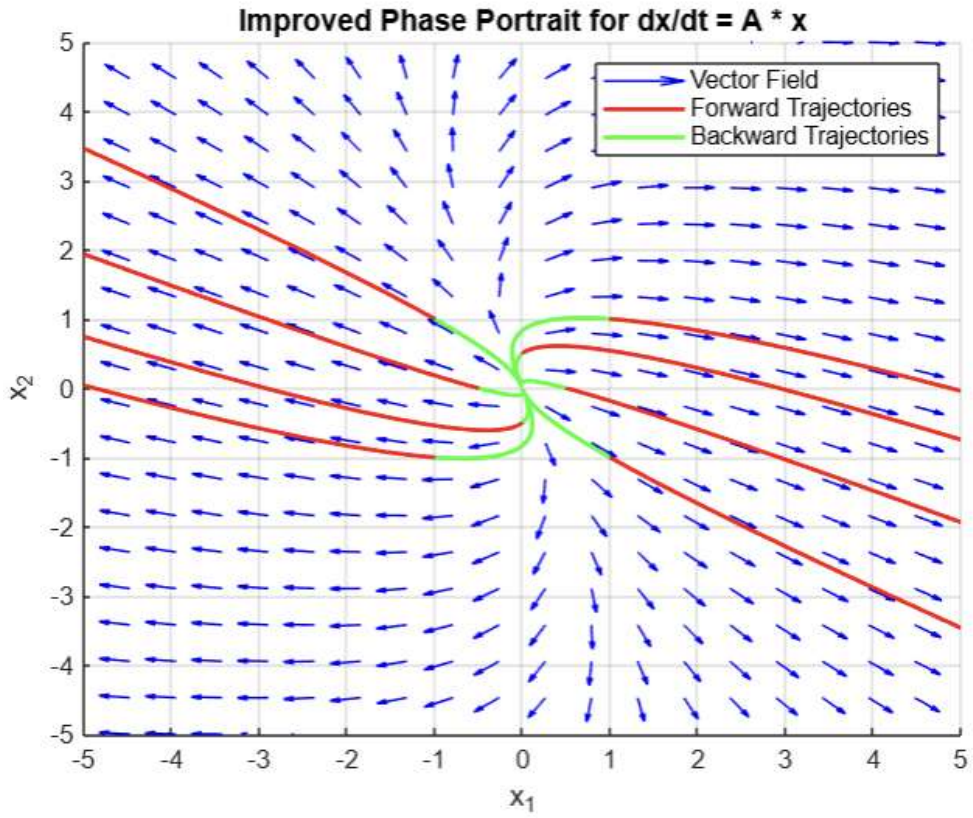


System is Marginally Stable (i.e. stable but not asymptotically)

OUTPUT :: 2::

```
A = 2×2
     9     2
    -3     2
```

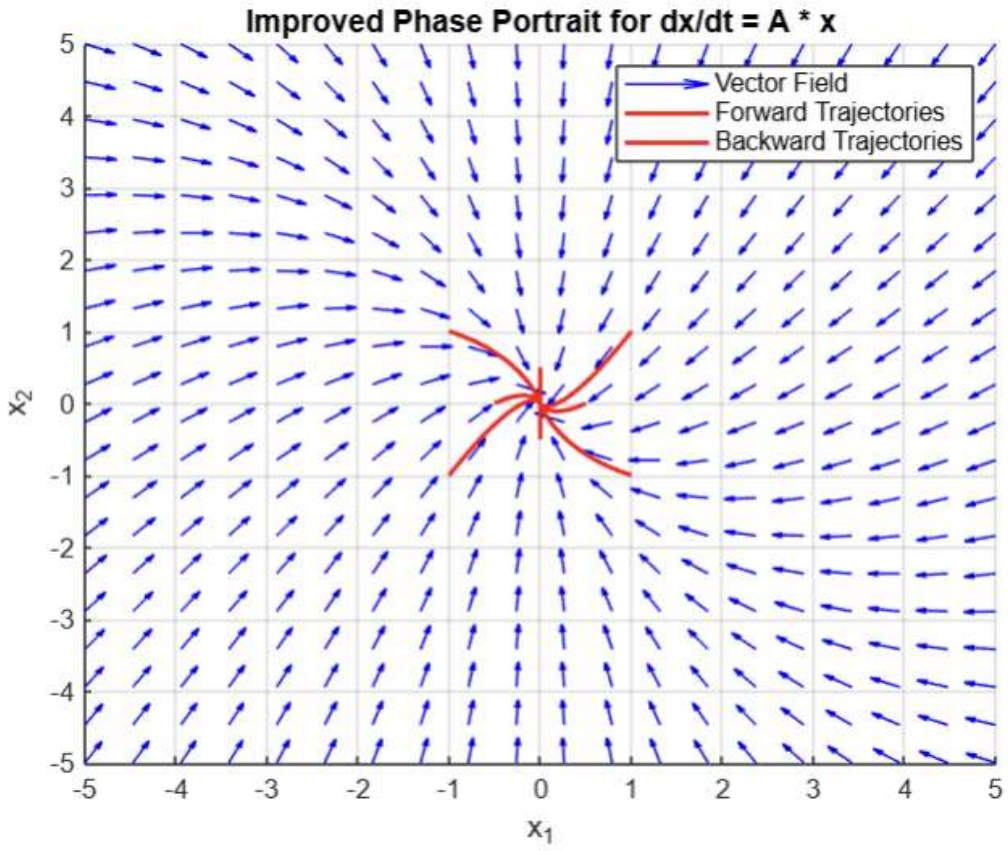
```
eigenvalues = 2×1
     8
     3
```



System is Unstable

OUTPUT :: 3::

```
A = 2×2
     -5     0
     -3    -5

eigenvalues = 2×1
     -5
     -5
```



System is Stable (Asymptotically Stable)

Experiment -5

AIM : Write a program to solve Lagrange's equation using Lagrange's method.

THEORY :

✅ **General Form:**

$$P(x,y,z)\frac{\partial z}{\partial x} + Q(x,y,z)\frac{\partial z}{\partial y} = R(x,y,z)$$

This is a **first-order linear PDE** in three variables x, y, z.

To solve it, we use:

$$\frac{dx}{P} = \frac{dy}{Q} = \frac{dz}{R}$$

These are called the **auxiliary equations**, and we solve them using the method of characteristics.

CODE :

```
syms x y z p q dx dy c1 c2


lhs=((y^2)*z/x)*p+(x*z)*q

rhs=(y^2); %RHS Side


C=coeffs(lhs,[q p]); %Coefficients of equation

P=C(1); %Extracting value of P

Q=C(2); %Extracting value of Q

R=rhs; %Extracting value of R

P=P*x/z;

Q=Q*x/z;

U = int(P,y) == int(Q,x) + c1; %Solving for U

S = int(P,y) - int(Q,x);

U = simplify(S)
```

```
P=y^2*z/P;
R=y^2*z/R;
V = int(P,x) == int(R,z) + c2; %Solving for V
T = int(P,x) - int(R,z);
V = simplify(T)
disp("Solution is given as f(U,V): ")
disp('f(')
disp(U)
disp(', ')
disp(V)
disp(') == 0')
```

OUTPUT :

```
lhs =
```

$$q\,x\,z+\frac{p\,y^2\,z}{x}$$

```
rhs =
```

$$y^2$$

```
U =
```

$$\frac{y^3}{3}-\frac{x^3}{3}$$

```
V =
```

$$\frac{z\,(2\,x-z)}{2}$$

```
Solution is given as f(U,V):
f(
```

$$\frac{y^3}{3}-\frac{x^3}{3}$$

```
,
```

$$\frac{z\,(2\,x-z)}{2}$$

```
) == 0
```

Experiment -6

AIM : Write a program to solve non-linear PDE using Charpit's Method

THEORY :

Charpit's method is used to find the **complete integral** of a **nonlinear first-order PDE** of the form:

$$F(x, y, z, p, q) = 0$$

Where:

- $p = \frac{\partial z}{\partial x}$
- $q = \frac{\partial z}{\partial y}$

This method helps us solve nonlinear PDEs that cannot be handled by Lagrange's method.

✅ Charpit's Auxiliary Equations

To solve the PDE, we derive the **Charpit's equations**:

$$\frac{dx}{\frac{\partial F}{\partial p}} = \frac{dy}{\frac{\partial F}{\partial q}} = \frac{dz}{p\frac{\partial F}{\partial p} + q\frac{\partial F}{\partial q}} = \frac{dp}{-\frac{\partial F}{\partial x} - p\frac{\partial F}{\partial z}} = \frac{dq}{-\frac{\partial F}{\partial y} - q\frac{\partial F}{\partial z}}$$

You then integrate these **characteristic equations** to find x, y, z, p, q.

CODE :

```
syms x y z p q dx dy c1 c2 a b


lhs = 2*q*(z-(p*x)-(q*y));

rhs = 1+(q*q);


% Given equation

f = lhs - rhs


% Finding partial derivatives
```

```
fx = diff(f,x);
fy = diff(f,y);
fz = diff(f,z);
fp = diff(f,p);
fq = diff(f,q);

% denominator of dp, dq, dz
f1 = fx + p* fz
f2 = fy + q * fz
f3 = simplify(-p*fp-q*fq)

% Solving for given condition
if f2==0 %condtion for q=constant
   Q = a
   eqn = subs(f, q, Q);
   eq2 = solve(eqn, p);
   P = eq2
elseif f1==0 %condtion for p=constant
   P = a
   eqn = subs(f, p, P);
   eq2 = solve(eqn, q);
   Q = eq2
end
sol=int(Q,y)+int(P,x);
disp("Solution is given as f(x,y)= ")
disp(sol)
```

OUTPUT :: 1::

eq = $-2\,q\,(p\,x - z + q\,y) = q^2 + 1$

f1 = 0

f2 = 0

f3 = $2\,q\,(q - z + 2\,p\,x + 2\,q\,y)$

Q = a

P =

$$-\frac{a^2 - 2\,a\,(z - a\,y) + 1}{2\,a\,x}$$

Solution is given as f(x,y)=

$$a\,y - \frac{\log(x)\,(2\,a^2\,y - 2\,a\,z + a^2 + 1)}{2\,a}$$

OUTPUT :: 2::

eq = $z^2 = p\,q\,x\,y$

f = $z^2 - p\,q\,x\,y$

f1 = $2\,p\,z - p\,q\,y$

f2 = $2\,q\,z - p\,q\,x$

f3 = $2\,p\,q\,x\,y$

Solution is given as f(x,y)=

$$a\,y - \frac{\log(x)\,(2\,a^2\,y - 2\,a\,z + a^2 + 1)}{2\,a}$$

Experiment -7

AIM : Write a program in MATLAB to solve higher order linear homogeneous partial differential equations with constant coefficients.

THEORY :

- **General Form:**

A higher-order linear homogeneous PDE with constant coefficients can be written as:

$$a_1 \frac{\partial^n u}{\partial x^n} + a_2 \frac{\partial^m u}{\partial t^m} + \cdots = 0$$

where:

- $u(x,t)$ is the unknown function,
- $a_1, a_2, \ldots$ are constant coefficients,
- The equation involves higher-order derivatives with respect to space and/or time.

CODE :

```
syms D d x y z m f1 f2 f3

F = (D*D*D - 6*D*D*d + 11*D*d*d - 6*d*d*d)*z;

f = exp(5*x + 6*y);

eq = F==f


eqn1 = subs(F, D, m);

eqn2 = subs(eqn1, d, 1)


mval = solve(eqn2, m)


if mval(1)~=mval(2)~=mval(3)
   CF = f1*(y+mval(1)*x) + f2*(y+mval(2)*x) + f3*(y+mval(3)*x)
elseif mval(1) == mval(2) ~= mval(3)
   CF = f1*(y+mval(1)*x) + x*f2*(y+mval(2)*x) + f3*(y+mval(3)*x)
```

```
elseif mval(1) ~= mval(2) == mval(3)
   CF = f1*(y+mval(1)*x) + x*f2*(y+mval(2)*x) + f3*(y+mval(3)*x)
elseif mval(1) == mval(3) ~= mval(2)
   CF = x*f1*(y+mval(1)*x) + f2*(y+mval(2)*x) + f3*(y+mval(3)*x)
else
   CF = f1*(y+mval(1)*x) + x*f2*(y+mval(2)*x) + x*x*f3*(y+mval(3)*x)
end
F1 = F/z;
F2 = f;
fpi = log(f);
valx = diff(fpi,x)
valy = diff(fpi,y)
while true
   eqn3 = subs(F1,D,valx);
   eqn4 = subs(eqn3,d,valy);
   if eqn4~=0
      break
   else
      F1 = diff(F1,D);
      F2 = F2*x;
   end
end
PI = F2/eqn4
final_ans = CF+PI
```

OUTPUT :: 1::

eq = $z\,(\mathrm{D}^3 - 6\,\mathrm{D}^2\,d + 11\,\mathrm{D}\,d^2 - 6\,d^3) = \mathrm{e}^{5\,x+6\,y}$

eqn2 = $z\,(m^3 - 6\,m^2 + 11\,m - 6)$

mval =

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

CF = $f_1\,(x+y) + f_2\,(2\,x+y) + f_3\,(3\,x+y)$

valx = 5

valy = 6

PI =

$$-\frac{\mathrm{e}^{5\,x+6\,y}}{91}$$

final_ans =

$$f_1\,(x+y) - \frac{\mathrm{e}^{5\,x+6\,y}}{91} + f_2\,(2\,x+y) + f_3\,(3\,x+y)$$

OUTPUT :: 2::

eq = $-z\,(-6\,\mathrm{D}^3 + 6\,\mathrm{D}\,d^2 + 6\,d^3) = \mathrm{e}^{6\,y^2+3\,x\,y}$

eqn2 = $-z\,(-6\,m^3 + 6\,m + 6)$

mval =

$$\begin{pmatrix} \mathrm{root}(z^3 - z - 1, z, 1) \\ \mathrm{root}(z^3 - z - 1, z, 2) \\ \mathrm{root}(z^3 - z - 1, z, 3) \end{pmatrix}$$

CF = $f_1\,(y + x\,\mathrm{root}(z^3 - z - 1, z, 1)) + f_2\,(y + x\,\mathrm{root}(z^3 - z - 1, z, 2)) + f_3\,(y + x\,\mathrm{root}(z^3 - z - 1, z, 3))$

valx = $3\,y$

valy = $3\,x + 12\,y$

PI =

$$-\frac{\mathrm{e}^{6\,y^2+3\,x\,y}}{18\,y\,(3\,x+12\,y)^2 + 6\,(3\,x+12\,y)^3 - 162\,y^3}$$

final_ans =

$$f_1\,(y + x\,\mathrm{root}(\sigma_1, z, 1)) + f_2\,(y + x\,\mathrm{root}(\sigma_1, z, 2)) + f_3\,(y + x\,\mathrm{root}(\sigma_1, z, 3)) - \frac{\mathrm{e}^{6\,y^2+3\,x\,y}}{18\,y\,{\sigma_2}^2 + 6\,{\sigma_2}^3 - 162\,y^3}$$

where

$\sigma_1 = z^3 - z - 1$

$\sigma_2 = 3\,x + 12\,y$

Experiment -8

AIM : Write a program in MATLAB to solve one-dimensional heat equation using method of separation of variables.

THEORY :

We are solving the heat equation:

$$\frac{\partial u}{\partial t} = \alpha^2 \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < L, \quad t > 0$$

with boundary conditions:

$$u(0,t) = u(L,t) = 0$$

and initial condition:

$$u(x,0) = f(x)$$

We'll assume a standard solution of the form:

$$u(x,t) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha^2 \left(\frac{n\pi}{L}\right)^2 t}$$

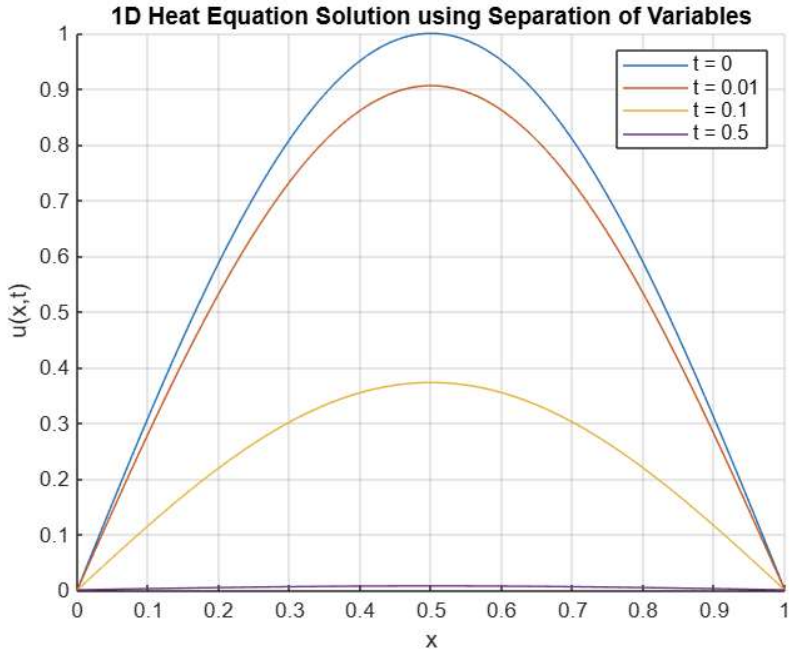
CODE :

```
L = 1;

alpha = 1;

N = 100;

x_points = 100;

t_values = [0, 0.01, 0.1, 0.5];

x = linspace(0, L, x_points);

f = @(x) sin(pi * x)


% Fourier coefficients B_n

B = zeros(1, N);
```

```
for n = 1:N
   integrand = @(x) f(x) .* sin(n * pi * x / L);
   B(n) = 2 / L * integral(integrand, 0, L);
end
integrand
figure;
hold on;
colors = lines(length(t_values));
legend_entries = cell(1, length(t_values));

for k = 1:length(t_values)
   t = t_values(k);
   u = zeros(size(x));
   for n = 1:N
      u = u + B(n) * sin(n * pi * x / L) * exp(-alpha^2 * (n * pi / L)^2 * t);
   end
   plot(x, u, 'DisplayName', ['t = ', num2str(t)], 'Color', colors(k,:));
   legend_entries{k} = ['t = ', num2str(t)];
end

xlabel('x');
ylabel('u(x,t)');
title('1D Heat Equation Solution using Separation of Variables');
legend show;
grid on;
```

OUTPUT :: 1::

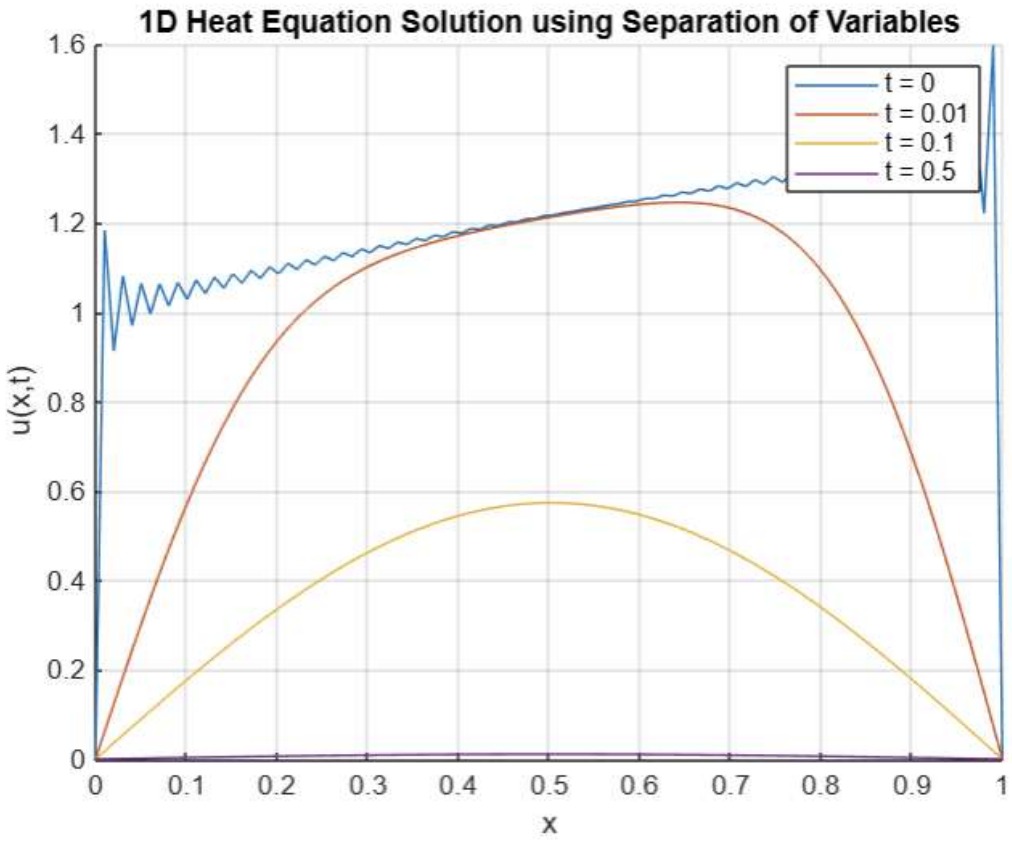
```
f = function_handle with value:
    @(x)sin(pi*x)
integrand = function_handle with value:
    @(x)f(x).*sin(n*pi*x/L)
 B = 1×100
      1.0000    0.0000    0.0000   -0.0000    0.0000   -0.0000   -0.0000    0.0000    0.0000    0.0000    0.0000   -0.0000   -0.0000   -0.0000    0.0000 ...

 u = 1×100
           0    0.0317    0.0634    0.0951    0.1266    0.1580    0.1893    0.2203    0.2511    0.2817    0.3120    0.3420    0.3717    0.4009    0.4298 ...

 u = 1×100
           0    0.0287    0.0575    0.0861    0.1147    0.1432    0.1715    0.1996    0.2275    0.2553    0.2827    0.3099    0.3367    0.3633    0.3894 ...

 u = 1×100
           0    0.0118    0.0236    0.0354    0.0472    0.0589    0.0705    0.0821    0.0936    0.1050    0.1163    0.1275    0.1385    0.1494    0.1602 ...

 u = 1×100
           0    0.0002    0.0005    0.0007    0.0009    0.0011    0.0014    0.0016    0.0018    0.0020    0.0022    0.0025    0.0027    0.0029    0.0031 ...
```



OUTPUT :: 2::

```
f = function_handle with value:
    @(x)sin(x/2)+cos(x/2)

integrand = function_handle with value:
    @(x)f(x).*sin(n*pi*x/L)
```

```
u = 1×100
         0    1.1840    0.9129    1.0812    0.9700    1.0651    0.9963    1.0634    1.0144    1.0667    1.0291    1.0722    1.0421    1.0788 ...

u = 1×100
         0    0.0617    0.1231    0.1839    0.2439    0.3027    0.3602    0.4161    0.4702    0.5223    0.5722    0.6199    0.6652    0.7081 ...

u = 1×100
         0    0.0181    0.0361    0.0541    0.0721    0.0900    0.1078    0.1255    0.1431    0.1605    0.1778    0.1949    0.2118    0.2285 ...

u = 1×100
         0    0.0004    0.0007    0.0011    0.0014    0.0017    0.0021    0.0024    0.0028    0.0031    0.0035    0.0038    0.0041    0.0044 ...
```



Experiment -9

AIM : Write a program in MATLAB to solve one-dimensional wave equation using method of separation of variables.

THEORY :

📘 Wave Equation

We solve the equation:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < L, \quad t > 0$$

with boundary conditions:

$$u(0,t) = u(L,t) = 0$$

and initial conditions:

$$u(x,0) = f(x), \quad \frac{\partial u}{\partial t}(x,0) = g(x)$$

The solution is given by:

$$u(x,t) = \sum_{n=1}^{\infty} \left[A_n \cos\left(\frac{n\pi ct}{L}\right) + B_n \sin\left(\frac{n\pi ct}{L}\right) \right] \sin\left(\frac{n\pi x}{L}\right)$$

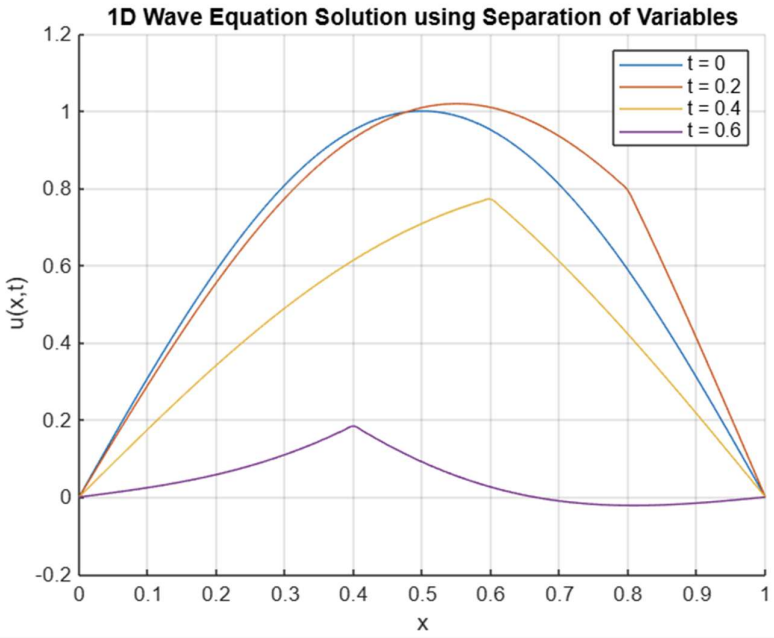
CODE :

```
L = 1;

c = 1;

N = 100;

x_points = 200;

t_values = [0, 0.2, 0.4, 0.6];

x = linspace(0, L, x_points);

f = @(x) sin(pi * x)

g = @(x) 2 * x

A = zeros(1, N);
```

```
B = zeros(1, N);
for n = 1:N
   integrand_A = @(x) f(x) .* sin(n * pi * x / L);
   A(n) = 2 / L * integral(integrand_A, 0, L);
   integrand_B = @(x) g(x) .* sin(n * pi * x / L);
   B(n) = 2 / (n * pi * c) * integral(integrand_B, 0, L);
end
figure;
hold on;
colors = lines(length(t_values));

for k = 1:length(t_values)
   t = t_values(k);
   u = zeros(size(x));
   for n = 1:N
      wn = n * pi * c / L;
      u = u + (A(n) * cos(wn * t) + B(n) * sin(wn * t)) .* sin(n * pi * x / L);
   end
   plot(x, u, 'DisplayName', ['t = ', num2str(t)], 'Color', colors(k,:));
end
xlabel('x');
ylabel('u(x,t)');
title('1D Wave Equation Solution using Separation of Variables');
legend show;
grid on;
```

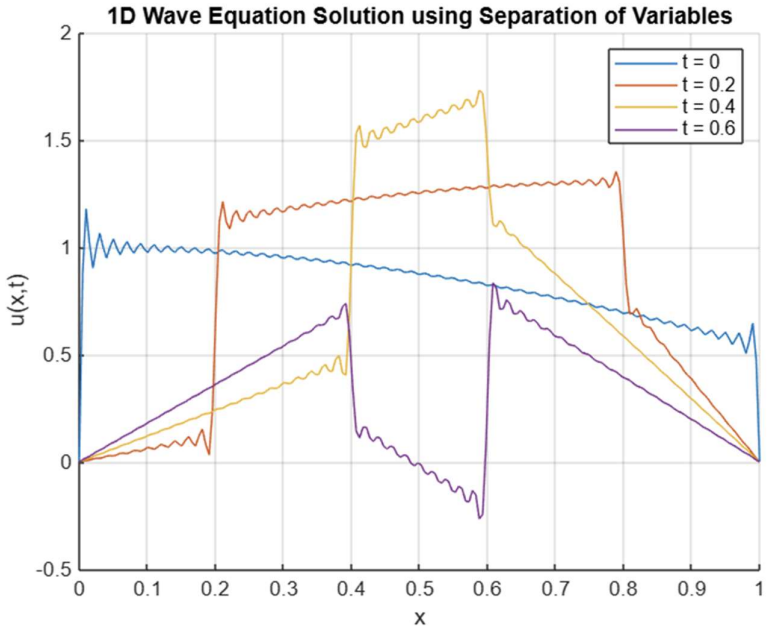
OUTPUT :: 1::

```
f = function_handle with value:
    @(x)sin(pi*x)
g = function_handle with value:
    @(x)2*x
```



OUTPUT :: 2::

```
f = function_handle with value:
    @(x)cos(x)
g = function_handle with value:
    @(x)3*x+sin(x)
```



Experiment -10

AIM : Write a program in MATLAB to solve Laplace equation using method of separation of variables.

THEORY :

We solve the **2D Laplace equation**:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

on a rectangular domain $0 < x < a, 0 < y < b$, with boundary conditions like:

- $u(0, y) = u(a, y) = 0$
- $u(x, 0) = 0$
- $u(x, b) = f(x)$, a known function

This is a classic case for using **separation of variables**.

📐 General Solution Using Separation of Variables

Assume:

$$u(x, y) = \sum_{n=1}^{\infty} A_n \sin\left(\frac{n\pi x}{a}\right) \sinh\left(\frac{n\pi y}{a}\right)$$

The coefficients A_n are found from:

$$A_n = \frac{2}{a \sinh\left(\frac{n\pi b}{a}\right)} \int_0^a f(x) \sin\left(\frac{n\pi x}{a}\right) dx$$

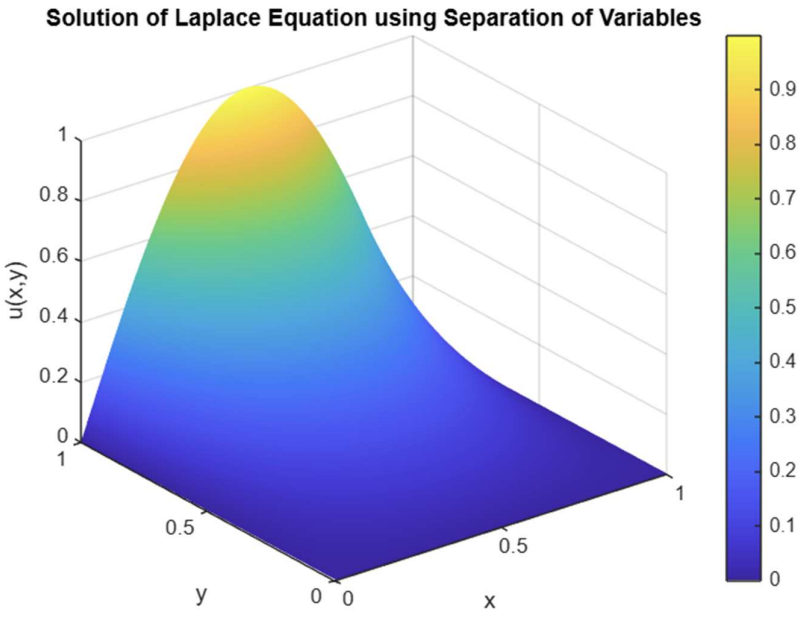
CODE :

```
a = 1;

b = 1;


Nx = 50;

Ny = 50;

x = linspace(0, a, Nx);
```

```
y = linspace(0, b, Ny);
[X, Y] = meshgrid(x, y);

N = 50;
f = @(x) sin(pi * x)

A = zeros(1, N);
for n = 1:N
   integrand = @(x) f(x) .* sin(n * pi * x / a);
   A(n) = 2 / (a * sinh(n * pi * b / a)) * integral(integrand, 0, a);
end

U = zeros(size(X));
for n = 1:N
   U = U + A(n) * sin(n * pi * X / a) .* sinh(n * pi * Y / a);
end

figure;
surf(X, Y, U);
xlabel('x');
ylabel('y');
zlabel('u(x,y)');
title('Solution of Laplace Equation using Separation of Variables');
shading interp;
colorbar;
```

OUTPUT :: 1::

```
f = function_handle with value:
    @(x)sin(pi*x)
```



OUTPUT :: 2::

```
a = 4
b = 2
f = function_handle with value:
    @(x)3*sin(x)+2*x-6
```